

mcp-repl

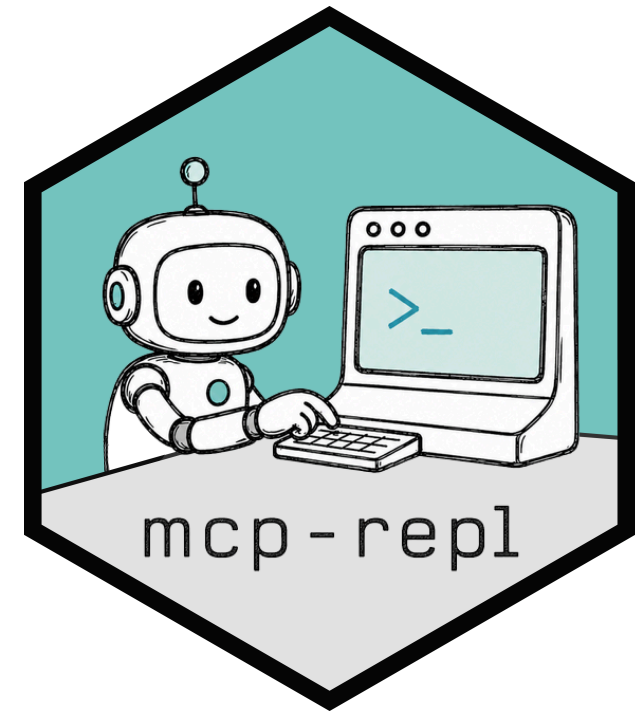
A Sandboxed R Runtime for LLM Agents

Give Agents a Real R Session

Tomasz Kalinowski

Posit Software PBC

useR! 2026, Warsaw



For agents, the standard interface to R is
Rscript

Explore this `sales.csv` dataset, tell me something new and interesting about it.



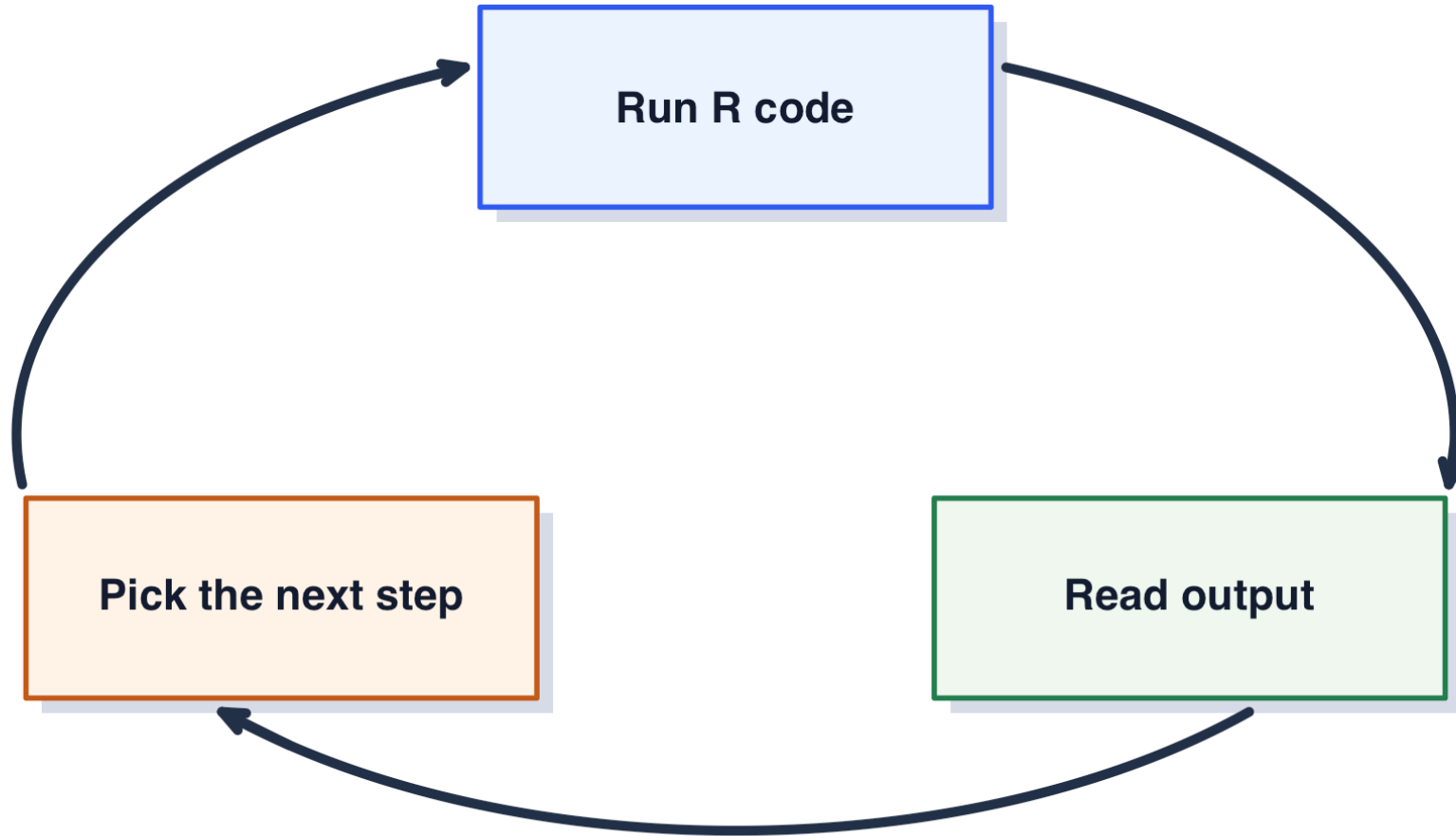
I'll inspect the data first, so I'll send R a quick one-off command.



bash()

```
Rscript -e "library(readr); library(dplyr);" df  
          <- read_csv('sales.csv'); glimpse(df); +  
                                summary(df$revenue)
```

Real work requires multiple turns



Each turn is a new R session

Explore this `sales.csv` dataset, tell me something new and interesting about it.



I'll inspect the data first, so I'll send R a quick one-off command.

 `bash()`

```
Rscript -e "library(readr); library(dplyr);" df
          <- read_csv('sales.csv'); glimpse(df); +
                                summary(df$revenue)
```

The result gives enough context to take the next step. This is turning into reusable setup, so I should move it into a script.

 `bash()`

```
cat <<'EOF' > /tmp/analysis.R
    library(readr)
    library(dplyr)
    df <- read_csv('sales.csv')
df <- filter(df, !is.na(revenue)) +
    print(glimpse(df))
    EOF
Rscript /tmp/analysis.R
```

Each turn is multiple tool calls

Explore this `sales.csv` dataset, tell me something new and interesting about it.



Now I'll iterate: change the file, run it, inspect the result, repeat.

 `bash()`

`Rscript /tmp/analysis.R +`

 `edit_file()`

`/tmp/analysis.R
Add model diagnostics. +`

 `bash()`

`Rscript /tmp/analysis.R +`

Using R through a shell tool

- The shell tool is available, so the agent uses it.
- Each turn, the R process starts from zero.
- Setup expressions repeat and compound, wasting time, compute, and tokens.
- The chat session has continuity, the R session does not.

mcp-repl

The name combines two acronyms:

MCP : Model Context Protocol

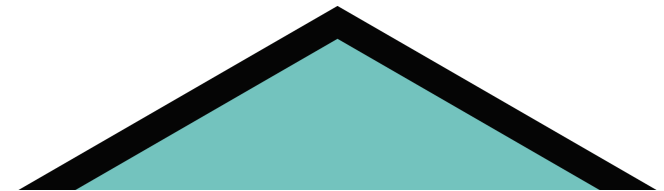
A standard way for an app to give a model tools

REPL : Read-Evaluate-Print loop

An interactive language session

mcp-repl

- An open-source stdio MCP server.
- A CLI binary built in Rust.
- Runs locally on your machine.
- Gives a live persistent runtime for an agent.
- Runs R or Python *in a sandbox*.



A compact, token-efficient interface

```
1 repl({  
2     "input": "1 + 1",  
3     "timeout_ms": 10000  
4 })
```

- One compact tool that accepts input code.
- Capabilities live in the runtime, not a wide MCP surface.

An interface designed for LLMs

```
1 repl({  
2     "input": "1 + 1",  
3     "timeout_ms": 10000  
4 })
```

Affordances for models:

- Help
- Plots
- Timeouts, interrupts, restarts
- Oversized outputs

Runs a regular R or Python interpreter

- Uses the same R and Python installation as you.
- Uses the same package libraries as you.
- Works with any standard CRAN, Bioconductor, or PyPI package.

Runs embedded R

- The main integration point is `R_ReadConsole`
- Unlike `eval(parse())`, it supports interactive modes:
 - debuggers: `browser()`, `recover()`, `pdb`
 - nested REPLs: `reticulate::repl_python()`, IPython
 - continuations and incomplete expressions
 - any interaction built on `readline()` or `input()`
- Unlike a PTY manager, it knows precisely when the runtime is ready for more input. No heuristics based on output or timing.

Sandbox model

- Broad runtime capabilities come with the risk of enormous harm.
- The sandbox is built with OS primitives (macOS, Linux, Windows)
 - Not an LLM prompt
 - Not a tool call filter
- Sandbox policy applies to the runtime process and all spawned child processes.

Default Sandbox Policy:

workspace-write

- No network access
- Filesystem edits restricted to the project directory, `tempdir()` and common cache locations (except `.git/`, `.agents/`, `.claude/`, `.codex/`)
- Most kernel and system calls are restricted
 - Narrow practical carveouts tailored for R code (e.g., `parallel::detectCores()`)

Optional sandbox policies

- `read-only`: no filesystem edits except `tempdir()`
- Additional writable roots
- Limited network access allowances via embedded proxy
- Read restrictions

Runs R with guardrails

- Runtimes with runaway memory consumption are killed before they run out of memory.
- On exit, `tempdir()` and spawned processes are cleaned up.
- A fresh session is automatically restarted
 - The REPL is always available

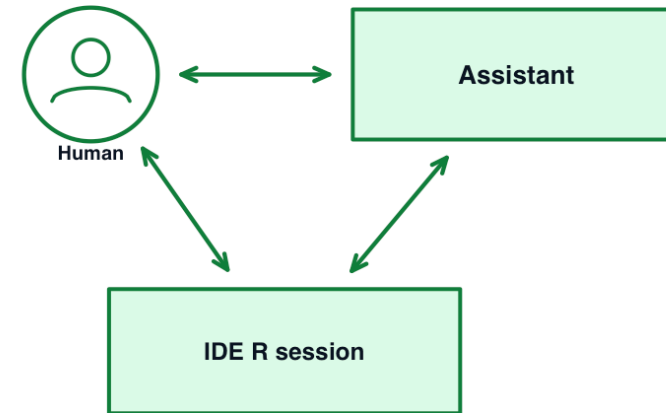
Two useful shapes

mcp-repl : agent-owned



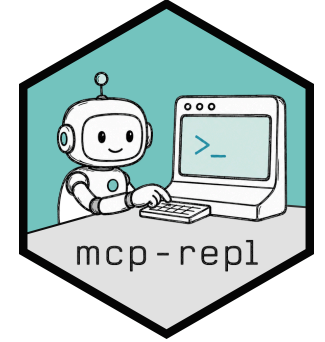
Agent can run unattended.
Human only interacts with the Agent.

Posit Assistant: Human in the loop



Human and agent share the same R Session.

Takeaways



Use `mcp-repl` to give an LLM agent:

- A real R or Python session
- A sandboxed runtime designed for agents
- A compact, token-efficient, full-featured workbench

<https://github.com/posit-dev/mcp-repl>

```
1 uv tool install posit-mcp-repl
2 mcp-repl install --client codex --interpreter r
```